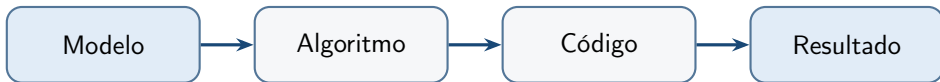


Conceptos Fundamentales en Programación

Variables, entrada/salida y operaciones en FORTRAN



Física Computacional – 106018C

Departamento de Física · Universidad del Valle

Semestre 2026-II

Objetivos de aprendizaje

- Explicar qué significa **programar** y reconocer los componentes básicos de un programa.
- Interpretar y construir **diagramas de flujo** sencillos.
- Reconocer variables y tipos fundamentales en **FORTRAN**.
- Usar `read` y `print` para entrada y salida.
- Aplicar operadores matemáticos, relacionales y lógicos.
- Adoptar buenas prácticas desde el primer programa.

Idea central

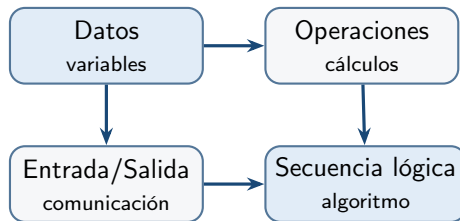
Antes de escribir código, debemos entender el problema y diseñar una secuencia lógica de pasos.

¿Qué es programar?

Programar significa expresar una solución mediante instrucciones que el computador puede ejecutar. Un programa combina

cuatro componentes fundamentales:

- datos,
- operaciones,
- decisiones,
- entrada y salida.

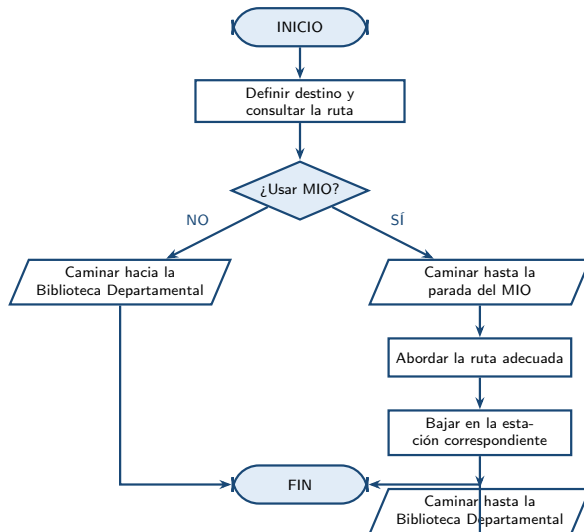


Primero diseñamos la solución; después la traducimos a FORTRAN.

Símbolos básicos de un diagrama de flujo

Símbolo	Nombre	Uso
	Terminal	Marca el inicio o el fin de un programa.
	Entrada/Salida	Lee información o muestra resultados.
	Proceso	Realiza asignaciones u operaciones.
	Decisión	Cambia el flujo según una condición.

Ejercicio 1: ir a la Biblioteca Departamental



Toda variable tiene

- un **nombre**,
- un **tipo**,
- un **valor**.

En FORTRAN se recomienda
usar siempre:

```
implicit none
```

para obligar a declarar todas
las variables.

integer
enteros

real
decimales

logical
verdadero/falso

character
un carácter

character(len=N)
texto

parameter
constante

Ejemplo: declaración y asignación

```
1 program variables
2   implicit none
3   integer :: x = 5
4   real :: y
5   character :: z = 'A'
6   y = 3.14
7   print *, "x = ", x
8   print *, "y = ", y
9   print *, "z = ", z
10 end program variables
```

Lectura del código

La declaración reserva memoria y define qué clase de información puede almacenar cada variable.

Constantes, precisión y cadenas

```
1 program mas_variables
2   use iso_fortran_env, only: real64
3   implicit none
4   integer, parameter :: MAX_INTENTOS = 3
5   real(real64), parameter :: PI = 3.141592653589793_real64
6   character(len=20) :: nombre
7   logical :: aprobado = .true.
8   nombre = "Ana"
9   print *, "Nombre: ", trim(nombre)
10  print *, "PI = ", PI
11  print *, "Aprobado: ", aprobado
12 end program mas_variables
```

parameter crea constantes y real64 permite controlar explícitamente la precisión numérica.

Entrada

Información que el programa recibe.

```
read *, variable
```

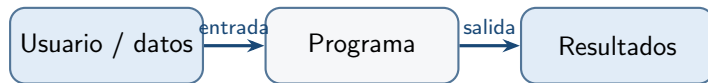
Ejemplos: teclado, archivo, sensor.

Salida

Información que el programa entrega.

```
print *, resultado
```

Ejemplos: pantalla, archivo, gráfica.



Ejemplo: entrada y salida

```
1 program entrada_salida
2   implicit none
3   integer :: edad
4   print *, "Ingrese su edad:"
5   read *, edad
6   print *, "En 2030 usted tendra ", edad + 4, " años."
7 end program entrada_salida
```

Observación

El programa se detiene en read hasta recibir un dato compatible con el tipo de la variable.

Ejemplo adicional: nombre y edad

```
1 program saludo
2   implicit none
3   character(len=30) :: nombre
4   integer :: edad
5   print *, "Cual es tu nombre?"
6   read '(A)', nombre
7   print *, "Cuantos años tienes?"
8   read *, edad
9   print *, "Hola ", trim(nombre), &
10      " , en 5 años tendrás ", edad + 5, " años."
11 end program saludo
```

El formato '(A)' permite leer una línea de texto que puede contener espacios.

Tipo	Operadores	Resultado
Matemáticos	+ - * / **	un valor numérico
Relacionales	== /= < > <= >=	.true. o .false.
Lógicos	.and. .or. .not. .neqv.	combinación de condiciones lógicas

Ejemplo

```
dentro_rango = (x > 0.0) .and. (x < 10.0)
```

Ejemplo: cálculo de la edad

```
1 program calcular_edad
2   implicit none
3   integer :: year, age
4   logical :: ya_cumplio
5   print *, "Digite su año de nacimiento:"
6   read *, year
7   print *, "Ya cumplio años en 2026? (T/F)"
8   read *, ya_cumplio
9   if (ya_cumplio) then
10      age = 2026 - year
11  else
12      age = 2026 - year - 1
13  end if
14  print *, "Tu edad es ", age, "."
15 end program calcular_edad
```

Aquí la lógica se expresa de manera explícita con `if...then...else`, más clara para una primera lectura.

Ejemplo adicional: operadores lógicos

```
1 program rango
2   implicit none
3   real :: x
4   logical :: dentro_rango
5   print *, "Ingrese un valor de x:"
6   read *, x
7   dentro_rango = (x > 0.0) .and. (x < 10.0)
8   print *, "x pertenece al intervalo (0,10)? ", dentro_rango
9 end program rango
```

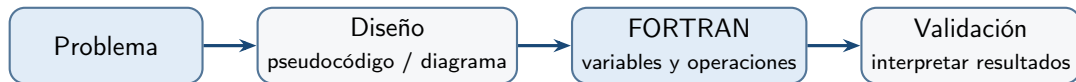
Interpretación

La expresión lógica será verdadera únicamente cuando ambas comparaciones sean verdaderas.

- Use siempre `implicit none`.
- Elija nombres descriptivos.
- Indente el código.
- Comente la intención, no lo evidente.
- Use constantes con `parameter`.
- Controle la precisión de los reales.
- Compile con advertencias activadas.
- Valide resultados con casos sencillos.

Compilación recomendada

```
gfortran -Wall -Wextra -fcheck=all programa.f90 -o programa
```



- Las variables tienen nombre, tipo y valor.
- `read` y `print` conectan el programa con el exterior.
- Los operadores transforman datos y permiten formular condiciones.
- La claridad del algoritmo es tan importante como la sintaxis.

1. Desarrolle el pseudocódigo y el diagrama de flujo para:
 - 1.1 sacar *Cien años de soledad* de la Biblioteca Central;
 - 1.2 preparar una tortilla;
 - 1.3 almorzar en la Cafetería Central.
2. Diseñe un programa que calcule la altura máxima de un proyectil, conociendo el ángulo respecto a la horizontal y la rapidez inicial. Incluya el diagrama de flujo y escriba el código en **FORTRAN**.
3. Elabore un glosario con las palabras resaltadas en itálicas en la Lectura 1.

Fin

Preguntas y discusión